Essay on Software Engineering for AI/AS

Gabriela Zarzar Gandler

2026

1 Introduction

Self-Supervised Learning (SSL) models for time series data learn representations from large datasets without label dependency. Fine-tuned on or adapted to an appropriate downstream task, these models can usually achieve state-of-the-art performance, often requiring less labeled data than supervised ones. However measuring the goodness of a learned representation remains a challenge, and usually requires substantial resources to train e.g. multiple linear probes with various labeled time series datasets.

My research lies in the intersection of representation learning and time series modeling, focusing on (a) studying label-free low-resource methods to estimate the quality of learned representations of sequential data, and particularly of time series data, and (b) studying ways to use such methods and the measures derived therefrom to better perform time series model fine-tuning and to develop suitable model adapters for reliable resource-aware model adaptation. There have been early works, which typically propose task-agnostic statistical measures of representation quality that predict generalization without explicit downstream task evaluation. Such works emerge from multiple domains, including the vision domain [1, 4, 5, 14], the speech domain [2, 16, 10], the text domain [9, 13], etc. However, despite some counterexamples [12, 15], relatively little research work has focused on time series model representations in particular.

My research aims to bridge this gap by studying (a) the conditions under which proposed measures are reliable and robust, (b) the strengths and weaknesses of the quality measures proposed in the literature, (c) novel methods to reliably assess the quality of time series models' learned representations, and (d) novel methods to integrate such measures in model finetuning and model adaptation.

2 Lecture Principles

I personally appreciated the fact that the paper Hidden Technical Debt in Machine Learning Systems [11] got special attention in the course. I have known this paper for so many years and have seen it mentioned in multiple presentations and conversations throughout my time in both academia and industry. As part of taking this WASP course, I got the opportunity to read the paper carefully and in detail, which I really enjoyed. During the time I had my first industry job, I remember acting and reasoning as if the ML algorithm and accuracy were enough to generate business value. I crafted the best image segmentation solution the company had ever seen, however, since this was the first ML project in the company, it was a significant challenge to integrate this solution in the product. It turns out the deployment was only completed after around 2 years from the date the ML model was developed.

From this experience I learned that ML systems entail way more dimensions than I anticipated: in particular, I learned that in order to create successful lasting applications of ML I need to think more end-to-end, expand my knowledge in software engineering and keep the people working on other bits of the ML system close to me throughout the project. Connecting to the paper, I find this *modus operandi* helpful and contributing to prevent the materialization of the technical debts the paper lists.

My own research is done within an Industrial PhD program, in a tech organization which is relatively mature in its software engineering practices. Generally, senior engineers in the organization are aware of the most common ML technical debts and have built tools and ways of working that prevent such debts from incurring. Despite these harnesses, I notice in my daily work in the applied ML research team how easy it is to adopt a narrow view and consider only the conditions and requirements relevant to the ML model development, while not giving sufficient attention to the bigger picture.

During the course, Robert briefly mentioned a paper named Technology Readiness Levels for ML Systems [7], which I personally found insightful. Besides proposing an organized framework of stages of readiness levels, the paper also emphasizes a workflow in which ML development is an iterative process rather than a sequential process, and emphasizes the integration of ML with a much wider system of software, hardware, data and people. I find this relevant to my own research work in my team, since it offers a framework for us to categorize our projects and understand the prerequisites to graduate projects to the next readiness level. A similar framework for Data Readiness Levels [8] has been very useful in multiple projects I conducted in the past, especially for the adoption of a common vocabulary across collaborators and stakeholders.

3 Guest Lecture Principles

One sentence that somehow got stuck in my mind from the guest lecture was that less routine work is not automatically better work. In the computer science universe, I personally often ”smell” an unspoken rule claiming that routine is by all means supposed to be automated. I personally like the fact that Per brought this topic to the lecture, and I would argue that it is more relevant than ever to reflect over what human beings need in order to be happy at work, to keep their motivation and to thrive. In the context of my research work, I notice how certain little routines help me get into a task and build flow, while other routines I gladly attempt to automate. My point is that I think individuals should be conscious and honest to themselves about what they truly believe should be automated and what shouldn’t.

Another sentence that got stuck in my mind was that change becomes a persistent condition. I would argue that this applies to software engineers, but also to many other professionals, including researchers. Lately I heard from an academic mentor that the way PhD students conduct research is substantially changing, and that PhD students should aim to become great ”prompt engineers”, for optimal AI assistance performance in research. While I do gladly recognize the support one can get with the right usage of AI agents, I also believe that it can be a challenge in itself to integrate the changing tools and changing workflows that emerge therefrom.

4 Data Scientists, Software Engineers, and AI Engineers - book chapters

The book chapters [6] describe data scientists as being mostly focused on the data (including data statistics and data patterns) and on the model development (including tasks such as feature engineering and hyperparameter tuning). Based on my experience in different tech companies, I would argue that the work scope of a data scientist can vary substantially from company to company. Typically in large tech companies data scientists have a somewhat well defined work scope that aligns well with this description. However in small companies the work scope can become quite blurred, as data scientists typically need to embrace responsibilities beyond model development, such as prototyping, application development and at times even integrations, requiring knowledge on ML infrastructure, product platform and data pipelines. However regardless of the work scope I think all data scientists need to possess a profound understanding of the data they deal with, and know how to exploit the data statistics.

In the book the software engineers are deemed to have a more high-level view of ML systems, where an ML model is only one component in a large (often complex) ML system. I agree with this distinction, and would add that my previous comment somewhat applies to software engineers too - I mean that the

larger the company is, the more specialised the software engineer might be into one component or a small set of components of the entire ML system.

I believe that the more software engineers and data scientists learn about each others' skills the better, since this way they are more capable of thinking end-to-end and understanding the requirements of different components of an ML system. I would argue that the increasing adoption of agentic workflows and AI coding assistants is already blurring the lines between these two disciplines, and in my opinion they can be used as good ways of learning skills beyond one's own discipline.

The chapters also argue that production ML requires a system-wide view, as opposed to only a model-centric view. My research is in its infancy stages now, hence a system-wide view doesn't apply at this point in time, apart from the fact that I know there is plenty of potential to apply my research findings in the product my company develops. Once I evolve in my experiments and mature my research questions, a system-wide view will become more relevant.

5 Paper Analysis

5.1 Paper 1

The paper Optimising for Energy Efficiency and Performance in Machine Learning [3] presents an argument I personally find important: that ML systems should be engineered around a measured trade-off between task performance and energy use. This contrasts with the common pure accuracy/latency optimisation problem that characterizes the problem definition in the vast majority of ML systems. The paper introduces ECOpt, an open-source framework that uses multi-objective Bayesian optimisation to tune model hyperparameters and produce a Pareto frontier, i.e. the configurations for which improving task accuracy would require using more energy, and vice versa.

I think there are a couple of core ideas from this paper worth mentioning. Firstly, it gives more importance to measuring energy consumption during inference instead of during training. It highlights that prior work often focuses on training energy, however inference energy can dominate lifetime compute cost. Secondly, the study measures energy directly as opposed to using parameter counts and FLOPs, which have shown not to be reliable proxies for inference energy, especially for CNNs and under-utilised hardware. The authors advocate publishing energy metrics alongside model metrics, for a more comprehensive model evaluation, which I find a great nudge as it offers particularly useful data for practitioners. Thirdly, the authors encourage optimisation of the whole deployment, since energy depends on workload and hardware configuration, and not only architecture. Fourthly, the authors carefully keep the trade-off explicit, rather than keeping energy and accuracy "inside" one arbitrary score, by offering the Pareto frontier, which lets an engineering team, for instance, select the lowest-energy model that still meets a product performance requirement. Fifthly and finally, the authors came to the conclusion that human intuition concerning hardware can be wrong, since smaller models may be more energy-efficient on CPUs than GPUs when the GPU is not saturated, and transformer energy efficiency tends to decline with model size in the performed experiments.

This topic matters for AI engineering, because it presents energy as a first-class system quality attribute, alongside attributes such as accuracy, latency and cost. In practical terms, the study supports model architecture selection, capacity planning, batch-size tuning, i.e. essential stages of ML system building, while helping software engineering teams avoid selecting models that look efficient but that are actually inefficient in the actual production stack. The key engineering lesson I extract from this paper is: optimize and report measured end-to-end inference energy in the target environment, as opposed to relying solely on abstract compute metrics. This is in my opinion both very valuable and tangible - hence I would argue this paper matters to the engineering of AI systems.

A large AI-intensive project close to my day-to-day work is the Candy Crush mobile game developed by King, where I work. The paper's ideas, as well as the open-source implementation of ECOpt as a

reusable Python framework, could be applied by multiple teams of ML engineers and data scientists at King. In my view, the current software engineering practices in the company do not sufficiently take energy optimization into account. The current energy saving practices are mostly related to cost saving initiatives (for e.g. compute and storage) in the cloud platform we use. This means there is room for improvement. Additionally, I personally find it sound to increase the share of responsability lieing on the shoulders of ML engineers and data scientist, since they could be more accountable for decreasing company's energy consumption. The paper's scope however doesn't include validating the methodology by using ECOpt in optimisation of large-scale models in close-to-production environments. Hence it would be beneficial to see an extension of the work in this direction.

In my own research I see the integration of ECOpt as an enhancement of my software engineering practices. Currently my colleagues and I apply neural architecture search in our projects, without taking reliable energy cosumption metrics into account, so I am positive towards accommodating ECOpt in our AI engineering workflow. Additionally I believe integrating ECOpt should also be feasible and low-effort in our case, since we already use MLFlow for experiment tracking, and the tool relies on MLFlow too.

5.2 Paper 2

The paper Investigating Issues that Lead to Code Technical Debt in Machine Learning Systems [17] identifies practices in ML development that create code technical debt, i.e. shortcuts that speed up initial delivery but make systems harder and more costly to maintain or change later. The core idea of the paper is to treat technical debt as an ML engineering problem across the ML workflow (as opposed to vaguely treating it as "messy code").

This study entails three research questions, namely (a) What are potential issues that could lead to ML code technical debt?, (b) To what degree do the identified issues manifest in practice, contributing to code-related technical debt?, (c) How do practitioners perceive the relevance of the issues in terms of leading to technical debt? The authors derived in total 34 potential issues causing debt, from common ML activities, then validated them in focus groups with nine experienced practitioners. They ended up retaining 30 issues, of which 24 were considered highly relevant.

The authors found out that the largest concentration of issues is in the data preprocessing phase. Typical preprocessing activities include identifying and handling outliers and missing values, selecting features, rebalancing and scaling the data, and removing inconsistent data. These design choices become embedded in pipelines, and when model quality later degrades, teams typically add patches instead of fixing the underlying data logic. Other major sources of issues identified by this work are incomplete or incorrect data integration/consumption, poor train–test–validation splits, skipped or weak hyperparameter tuning and algorithm testing, and evaluation metrics that do not reflect real project goals.

As discussed in the lectures, this topic matters to the engineering of AI systems. The paper offers a practical checklist that can be used across multiple phases in the ML workflow, aiding practitioners towards developing robust software solutions. I find it useful that the authors emphasize how model reliability and code maintainability are somewhat inseparable, making it clear that e.g. a data shortcut might later force pipeline rewrites, retraining and fragile workarounds.

As mentioned above, a large AI-intensive project close to my day-to-day work is the Candy Crush mobile game developed by King. The ideas in the paper and particularly the awareness of ML technical debt are to a good extent an integral part of software engineering practices being applied in the day-to-day work of multiple development teams across the organization. I like to say that King is a relatively data mature company, since it has been investing on robust software engineering practices for more than 10 years, which has become an essential foundation, on top of which there exist currently deployments of multiple ML solutions in the product, impacting the game experience of millions of players.

This paper relates to my own research, because the more aware I become about technical debt in ML

systems the more capable I become to avoid having issues silently embedded in my work deliverables. As an industrial PhD student, I often see how much human effort can incur in applied research AI projects in which coding issues are not addressed early on and instead treated as an afterthought. I therefore encourage my team to make a conscious choice of writing reliable code from the start. A practice that has worked well for us is to adopt code repository templates at the start of a new project - the templates offer useful integrations and engineering controls that work out of the box, creating standardization of best practices and more opportunities of collaboration. This being said, the message I take from this paper is not to never take a shortcut, but rather to be conscious about when a shortcut is ok to take, and when it should be avoided due to incurring substantial costs in the medium/long term.

Note that this study is exploratory and its conclusions come from two focus groups and nine practitioners only, so I would argue a broader longitudinal validation is still necessary.

6 Evidence, originality, and GenAI transparency

The reference section at the end of this report contains all the sources used. The Chapters 2 and 3 are based on my own interpretation and notes during the in-person lectures in Gothenburg, as well as the lecture notes used during the course and available at the course Canvas page. Chapter 4 is based on a book [6]. Chapter 5 is based on two peer-reviewed and published recent long papers at CAIN conferences [3, 17]. I used Generative AI, and particularly ChatGPT, to search for the two CAIN papers mentioned in Chapter 5. The writing of the report is my own piece of work.

References

[1] Kumar K Agrawal, Arnab Kumar Mondal, Arna Ghosh, and Blake Richards. \alpha-req : Assessing representation quality in self-supervised learning by measuring eigenspectrum decay. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 17626–17638. Curran Associates, Inc., 2022.

[2] Zakaria Aldeneh, Vimal Thilak, Takuya Higuchi, Barry-John Theobald, and Tatiana Likhomanenko. Towards automatic assessment of self-supervised speech models using rank. In *ICASSP 2025 - 2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 1–5, 2025.

[3] Emile Dos Santos Ferreira, Andrei Paleyes, and Neil D. Lawrence. Optimising for energy efficiency and performance in machine learning. In *2026 IEEE/ACM 5th International Conference on AI Engineering – Software Engineering for AI (CAIN)*, 2026.

[4] Quentin Garrido, Randall Balestriero, Laurent Najman, and Yann Lecun. RankMe: Assessing the downstream performance of pretrained self-supervised representations by their rank. In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett, editors, *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 10929–10974. PMLR, 23–29 Jul 2023.

[5] Neha Kalibhat, Kanika Narang, Hamed Firooz, Maziar Sanjabi, and Soheil Feizi. Measuring self-supervised representation quality for downstream classification using discriminative features. In *Proceedings of the Thirty-Eighth AAAI Conference on Artificial Intelligence and Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence and Fourteenth Symposium on Educational Advances in Artificial Intelligence*, AAAI'24/IAAI'24/EAAI'24. AAAI Press, 2024.

[6] Christian Kästner. *Machine Learning in Production: From Models to Products*. MIT Press, Cambridge, MA, USA, April 2025.

[7] Alexander Lavin, Ciarán M. Gilligan-Lee, Alessya Visnjic, Siddha Ganju, Dava Newman, Sujoy Ganguly, Danny Lange, Atílím Güneş Baydin, Amit Sharma, Adam Gibson, Stephan Zheng, Eric P. Xing, Chris Mattmann, James Parr, and Yarin Gal. Technology readiness levels for machine learning systems. *Nature Communications*, 13(1), October 2022.

[8] Neil D. Lawrence. Data readiness levels. *CoRR*, abs/1705.02245, 2017.

[9] Melody Zixuan Li, Kumar Krishna Agrawal, Arna Ghosh, Komal Kumar Teru, Adam Santoro, Guillaume Lajoie, and Blake A. Richards. Tracing the representation geometry of language models from pretraining to post-training. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[10] Alexander H. Liu, Sung-Lin Yeh, and James R. Glass. Revisiting self-supervised learning of speech representation from a mutual information perspective. In *ICASSP 2024 - 2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 12051–12055, 2024.

[11] D. Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-François Crespo, and Dan Dennison. Hidden technical debt in machine learning systems. In C. Cortes, N. Lawrence, D. Lee, M. Sugiyama, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 28. Curran Associates, Inc., 2015.

[12] Rashed Shelim, Shengzhe Xu, Walid Saad, and Naren Ramakrishnan. Is isotropy a good proxy for generalization in time series forecasting with transformers? *Transactions on Machine Learning Research*, 2025.

[13] Oscar Skean, Md Rifat Arefin, Dan Zhao, Niket Patel, Jalal Naghiyev, Yann LeCun, and Ravid Shwartz-Ziv. Layer by layer: uncovering hidden representations in language models. In *Proceedings of the 42nd International Conference on Machine Learning*, ICML'25. JMLR.org, 2025.

[14] Anton Tsitsulin, Marina Munkhoeva, and Bryan Perozzi. Unsupervised embedding quality evaluation. In Timothy Doster, Tegan Emerson, Henry Kvinge, Nina Miolane, Mathilde Papillon, Bastian Rieck, and Sophia Sanborn, editors, *Proceedings of 2nd Annual Workshop on Topology, Algebra, and Geometry in Machine Learning (TAG-ML)*, volume 221 of *Proceedings of Machine Learning Research*, pages 169–188. PMLR, 28 Jul 2023.

[15] Songkang Wen, Vasilii Feofanov, and Jianfeng Zhang. Measuring pre-training data quality without labels for time series foundation models, 2024.

[16] Ryan Whetten, Lucas Maison, Titouan Parcollet, Marco Dinarelli, and Yannick Estève. Towards Early Prediction of Self-Supervised Speech Model Performance. In *Interspeech 2025*, pages 1228–1232, 2025.

[17] Rodrigo Ximenes, Antonio Pedro Santos Alves, Tatiana Escovedo, Rodrigo Spinola, and Marcos Kalinowski. Investigating issues that lead to code technical debt in machine learning systems. In *2025 IEEE/ACM 4th International Conference on AI Engineering – Software Engineering for AI (CAIN)*, pages 173–183, 2025.